

Advanced SIMD Optimizations

Mainack Mondal

Sandip Chakraborty

CS 60203

Autumn 2026



Outline

- SSE Memory Operations
 - Kinds of Memory operations
 - Memory Alignment
 - Pointer Aliasing
- GCC Auto Vectorization
- Fused-Multiply-Add (FMA)
 - What is FMA? Why FMA?
 - Family of FMA
 - Advantages
- Advanced Matrix Extensions (AMX)
 - What is AMX? Why Matrices?
 - Tiles, Registers & TMUL
 - AMX vs AVX-512 / FMA
- SIMD in Quant Trading Practice
 - Branchless Masking (AVX-512)
 - Horizontal Reductions
 - Gather / Scatter Lookups

SSE Memory Operations

(Slides partially taken from Marat Dukhan and Andreas Schmitz)

SSE Memory Operations

We have already seen two kinds of load/store instructions

- **Aligned** (eg: `_mm_load_ps`)
 - Mandate that pointer is aligned on 16-byte boundary
 - Pros: Faster loads/stores (why?)
 - Cons: Can cause segmentation faults if misaligned data is loaded
- **Mis-Aligned** (eg: `_mm_loadu_ps`)
 - Can work with any pointers
 - But, has computation overhead
 - Multiple reads necessary
 - Additional code to extract the data

Memory Alignment in C/C++

But ... How do you align memory?

The answer is : Using specific compiler directives

- Directives to control alignment behavior:
 - GCC specific [FSF15, 6.38]
 - `__attribute__((aligned (ALIGN)))`
 - `__attribute__((packed))`
 - C11 Standard [ISO 11, 6.2.8, 7.22.3]
 - `aligned_alloc(size_t alignment, size_t size)`
 - `_Alignas(expression)` and `_Alignas(type)`

Memory Alignment Examples

- `struct V{short s[3];} __attribute__((aligned(8)));`
 - size of V = 6 bytes + 2 bytes (padding)
- `char c[2] __attribute__((aligned(8)));`
 - size of c = 2 bytes + 6 bytes (padding)
- `struct A{char a; int b;} __attribute__((packed));`
 - size of A = 1 byte + 4 bytes (no padding because packed)

Pointer Aliasing

- Refers to memory addressed by different names
- Example: `char b; char *a = &b;`
- Needs to be considered by the compiler
- Can result in code overhead (will see in examples)

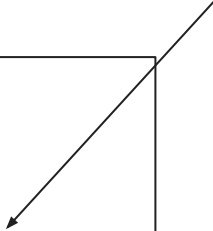
Pointer Aliasing Examples

```
void foo(int *a, int *b, int *c)
{
    *a = 42;
    *b = 23;
    *c = *a;
}
```

Assembly

```
mov DWORD PTR [rdi], 42
mov DWORD PTR [rsi], 23
mov edx, DWORD PTR [rdi]
mov DWORD PTR [rdx], edx
```

due to aliasing



Pointer Aliasing Examples

How do you stop aliasing? Use `restrict` keyword (C99 standard)

```
void foo(int * restrict a, int * restrict b, int *c)
{
    *a = 42;
    *b = 23;
    *c = *a;
}
```

Assembly →

```
mov DWORD PTR [rdi], 42
mov DWORD PTR [rsi], 23
mov DWORD PTR [rdx], 42
```

No aliasing

Pointer Aliasing - Remarks

- `restrict` needs to be used carefully
- Programmer is responsible for proper usage
- Mishandling can lead to wrong programs

Outline

- SSE Memory Operations
 - Kinds of Memory operations
 - Memory Alignment
 - Pointer Aliasing
- GCC Auto Vectorization
- Fused-Multiply-Add (FMA)
 - What is FMA? Why FMA?
 - Family of FMA
 - Advantages
- Advanced Matrix Extensions (AMX)
 - What is AMX? Why Matrices?
 - Tiles, Registers & TMUL
 - AMX vs AVX-512 / FMA
- SIMD in Quant Trading Practice
 - Branchless Masking (AVX-512)
 - Horizontal Reductions
 - Gather / Scatter Lookups

GCC Auto Vectorization

Auto-vectorization related Flags

- `-O -ftree-vectorize`
 - Activates auto-vectorization
- `-O3`
 - Optimizations including auto vectorization
- `-fopt-info-vec, -fopt-info-vec-missed`
 - List (not) vectorized loops + additional information
- `-march=native`
 - Use instructions supported by the local CPU
- `-falign-functions=32, -falign-loops=32`
 - Aligns the address of functions / loops to be a multiple of 32 bytes

GCC Auto-Vectorization Examples

Simple Loop

```
# define SIZE (1 << 16)
void simpleLoop ( double * a, double * b)
{
    for ( int i = 0; i < SIZE ; i++)
    {
        a[i] += b[i];
    }
}
```

Compiler Output:

optimized: loop vectorized using 32 byte vectors
optimized: loop versioned for vectorization because
of possible aliasing

compile this with:

```
-O -ftree-vectorize, -fopt-info-vec,  
-fopt-info-vec-missed, -march=native
```

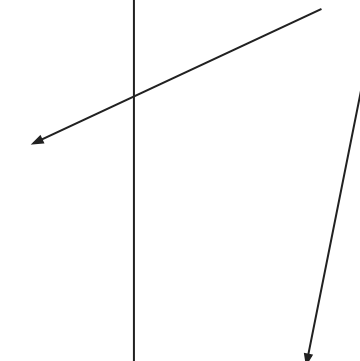
GCC Auto-Vectorization Examples (contd.)

Simple Loop (asm output)

.L3:

```
vmovupd    (%rdi,%rax), %ymm1
vaddpd     (%rsi,%rax), %ymm1, %ymm0
vmovupd    %ymm0, (%rdi,%rax)
addq       $32, %rax
cmpq       $524288, %rax
jne        .L3
ret
```

2 versions



.L2:

```
vmovsd     (%rdi,%rax), %xmm0
vaddsd     (%rsi,%rax), %xmm0, %xmm0
vmovsd     %xmm0, (%rdi,%rax)
addq       $8, %rax
cmpq       $524288, %rax
jne        .L2
ret
```

Let's make it better

GCC Auto-Vectorization Examples (Contd.)

Improved Loop

```
# define SIZE (1 << 16)
void simpleLoop ( double * restrict a, double * restrict b)
{
    for ( int i = 0; i < SIZE ; i++)
    {
        a[i] += b[i];
    }
}
```

Compiler Output:

optimized: loop vectorized using 32 byte vectors

GCC Auto-Vectorization Examples (contd.)

Improved Loop (asm output)

```
.L2:  
    vmovupd    (%rdi,%rax), %ymm1  
    vaddpd     (%rsi,%rax), %ymm1, %ymm0  
    vmovupd    %ymm0, (%rdi,%rax)  
    addq       $32, %rax  
    cmpq       $524288, %rax  
    jne        .L2  
    ret
```

No versions

Scope for further improvement ?

- Yes, it uses unaligned load/store

Let's make it even better

GCC Auto-Vectorization Examples (Contd.)

Optimised Loop

```
#define SIZE (1 << 16)
#define GCC_ALN(var, alignment) \
__builtin_assume_aligned(var, alignment)

void optimized_Loop(double *restrict a, double *restrict b)
{
    a = (double *)GCC_ALN(a, 32);
    b = (double *)GCC_ALN(b, 32);
    for (int i = 0; i < SIZE; i++)
    {
        a[i] += b[i];
    }
}
```

Compiler Output:

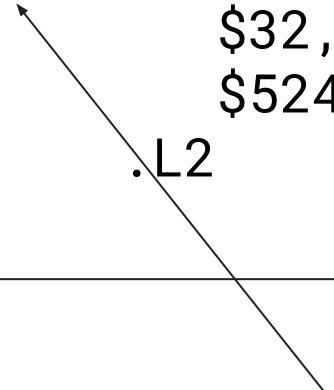
optimized: loop vectorized using 32 byte vectors

GCC Auto-Vectorization Examples (contd.)

Optimized Loop (asm output)

```
.L2:  
    vmovapd    (%rdi,%rax), %ymm1  
    vaddpd     (%rsi,%rax), %ymm1, %ymm0  
    vmovapd    %ymm0, (%rdi,%rax)  
    addq       $32, %rax  
    cmpq       $524288, %rax  
    jne        .L2  
    ret
```

aligned load/store



GCC Auto-Vectorization Examples (Contd.)

Optimised Loop (C11 Compatible)

```
#include <stdalign.h>
#define SIZE (1 << 16)

struct data {
    alignas(32) double vec[SIZE];
};

void optimized_Loop(struct data *restrict a, struct data *restrict b){
    for (int i = 0; i < SIZE; i++)
        a->vec[i] += b->vec[i];
}
```

Compiler Output:

optimized: loop vectorized using 32 byte vectors

Note: This produces the same assembly code as previous

Auto-vectorization Requirements and Limitations

- Countable loops
- No backward loop-carried dependencies
- No function calls : Except vectorizable math functions e.g. sin, sqrt,...
- Straight-line code (only one control flow: no switch)
- Loop to be vectorized must be innermost loop if nested

References: [Intel: requirements for vectorizable loops](#)

Outline

- SSE Memory Operations
 - Kinds of Memory operations
 - Memory Alignment
 - Pointer Aliasing
- GCC Auto Vectorization
- Fused-Multiply-Add (FMA)
 - What is FMA? Why FMA?
 - Family of FMA
 - Advantages
- Advanced Matrix Extensions (AMX)
 - What is AMX? Why Matrices?
 - Tiles, Registers & TMUL
 - AMX vs AVX-512 / FMA
- SIMD in Quant Trading Practice
 - Branchless Masking (AVX-512)
 - Horizontal Reductions
 - Gather / Scatter Lookups

FMA: Fused-Multiply-Add

What is FMA?

- Basically combining multiplication and Addition
 - computes $x*y + z$
- Introduced by IBM in their POWER Architecture in 1990s

FMA Family

- $\text{FMA}(x, y, c)$ - Fused Multiply-Add $\rightarrow x * y + c$
- $\text{FMS}(x, y, c)$ - Fused Multiply-Subtract $\rightarrow x * y - c$
- $\text{FNMA}(x, y, c)$ - Fused Negative Multiply-Add $\rightarrow -x*y + c$
- $\text{FNMS}(x, y, c)$ - Fused Negative Multiply-Subtract $\rightarrow -x * y - c$

Why FMA?

Let's see a comparison between FMA and Multiply-then-add:

- **Multiply + Add:**
 - First, $(x * y)$ is computed and rounded to double precision
 - Then, $(x * y) + c$ is computed and also rounded to double
- **Fused Multiply-Add:**
 - Single operation
 - Computed without intermediate rounding after multiplication

Advantages of FMA

- **Higher accuracy (How?)**
- **Higher performance**
 - Performs addition and multiplication at the cost of single multiplication
 - *AMD Bulldozer*: FP ADD/MUL/FMA latency = 5
 - Allows high-performance implementation of other floating-point algorithms eg: division, sqrt etc.

Resources on FMA

For further information see

- "Handbook of Floating Point Arithmetic" (2009), Muller, Brisebarre, de Dinechin, Jeannerod, Lefèvre
- "IA64 and Elementary Functions" (2000), Peter Markstein
- Wikipedia article on FMA: [Link](#)
- FMA Instruction set: [Link](#)

Outline

- **SSE Memory Operations**
 - Kinds of Memory operations
 - Memory Alignment
 - Pointer Aliasing
- **GCC Auto Vectorization**
- **Fused-Multiply-Add (FMA)**
 - What is FMA? Why FMA?
 - Family of FMA
 - Advantages
- **Advanced Matrix Extensions (AMX)**
 - What is AMX? Why Matrices?
 - Tiles, Registers & TMUL
 - AMX vs AVX-512 / FMA
- **SIMD in Quant Trading Practice**
 - Branchless Masking (AVX-512)
 - Horizontal Reductions
 - Gather / Scatter Lookups

AMX: Advanced Matrix Extensions

(Intel Xeon Scalable, Sapphire Rapids onward)

What is AMX?

- **AMX = Advanced Matrix Extensions, Intel's Sapphire Rapids Xeons (2023)**
 - Extends the “wider register” idea from SSE/AVX/AVX-512 to whole 2-D tiles
 - SSE: 128-bit vector. AVX-512: 512-bit vector. AMX: tile up to 16 rows × 64 bytes
- **Built for matrix multiply-accumulate**
 - The core op behind ML inference, risk models, PCA/regression in quant pipelines
 - One TMUL instruction $\approx$ thousands of scalar FMAs
- **Why quant / ML shops care**
 - Pricing models, factor regressions and small neural nets reduce to matrix multiplies
 - AMX runs that math in-core — no round trip to a discrete accelerator

AMX Tiles, Registers & TMUL

- **8 tile registers:** `tmm0` – `tmm7`
 - Each tile holds up to 16 rows × 64 bytes (1 KB) of data
- **Tile configuration**
 - `_tile_loadconfig()` sets rows / columns for every tile before use
- **Core instructions**
 - `_tile_loadadd(tile, base, stride)` – load a tile from memory
 - `_tile_dpbssd` / `_tile_dpbf16ps` – dot-product tile Multiply-Accumulate (int8 / bf16)
 - `_tile_store(tile, base, stride)` – store the result tile back to memory
- FMA computes $\mathbf{c} = \mathbf{a} * \mathbf{b} + \mathbf{c}$ on scalars/vectors; TMUL does the same on whole tiles in one instruction

AMX Code Example: Tile Matmul

```
#include <immintrin.h>

void amx_matmul(const int8_t *A, const int8_t *B,
int32_t *C) {
    __tilecfg cfg = {0};
    cfg.rows[0] = 16; cfg.colsb[0] = 64;
    cfg.rows[1] = 16; cfg.colsb[1] = 64;
    cfg.rows[2] = 16; cfg.colsb[2] = 64;
    _tile_loadconfig(&cfg);

    _tile_loadadd(0, A, 64);          // load A into tmm0
    _tile_loadadd(1, B, 64);          // load B into tmm1
    _tile_zero(2);                    // tmm2 = 0

    _tile_dpbssd(2, 0, 1);            // tmm2 += tmm0 * tmm1
    _tile_stored(2, C, 64 * sizeof(int32_t));

    _tile_release();
}
```

Compile with:
-mamx-tile -mamx-int8

One _tile_dpbssd = $16 \times 16 \times 64$
= 16,384 MACs in a single
instruction

AMX vs AVX-512 vs FMA

- **FMA (Part 2, Section 3):** one scalar/vector multiply-add per instruction (e.g. 8 doubles with AVX-512 FMA)
- **AVX-512:** 512-bit vector → 16 float FMAs per instruction
- **AMX:** a full tile Multiply-Accumulate → thousands of int8/bf16 MACs per instruction
- Rule of thumb (Sapphire Rapids): AMX can reach roughly 8× the peak int8/bf16 throughput of AVX-512 alone on large matrix workloads
- **Trade-off: AMX only helps for genuine matrix multiplies**
 - Plain SIMD (SSE/AVX/FMA) still wins for elementwise / streaming workloads — e.g. tick processing

Outline

- **SSE Memory Operations**
 - Kinds of Memory operations
 - Memory Alignment
 - Pointer Aliasing
- **GCC Auto Vectorization**
- **Fused-Multiply-Add (FMA)**
 - What is FMA? Why FMA?
 - Family of FMA
 - Advantages
- **Advanced Matrix Extensions (AMX)**
 - What is AMX? Why Matrices?
 - Tiles, Registers & TMUL
 - AMX vs AVX-512 / FMA
- **SIMD in Quant Trading Practice**
 - Branchless Masking (AVX-512)
 - Horizontal Reductions
 - Gather / Scatter Lookups

SIMD in Quant Trading Practice

(the kind of code that runs inside low-latency trading engines)

Branchless Filtering (AVX-512 Masks)

- A mispredicted `if` on nanosecond-scale tick data is a stall you can't afford
- AVX-512 mask registers (`k0–k7`): compare 8/16 values at once, get a bitmask — zero branches

```
#include <immintrin.h>

// keep prices > threshold, without branching
int filter_above(const double *prices, double threshold,
                double *out, int n) {
    __m512d thresh = _mm512_set1_pd(threshold);
    int count = 0;
    for (int i = 0; i < n; i += 8) {
        __m512d p = _mm512_loadu_pd(&prices[i]);
        __mmask8 mask = _mm512_cmp_pd_mask(p, thresh, _CMP_GT_OQ);
        _mm512_mask_compressstoreu_pd(&out[count], mask, p);
        count += _mm_popcnt_u32(mask);
    }
    return count;
}
```

Horizontal Reductions: Best Bid

- Order books need a fast “max across a vector” — the best bid
- `_mm512_reduce_max_pd` folds a 512 bit ZMM register to one scalar in hardware

```
#include <immintrin.h>

double best_bid(const double *bids, int n) {
    __m512d best = _mm512_set1_pd(-__builtin_inf());
    int i = 0;
    for (; i + 8 <= n; i += 8) {
        __m512d v = _mm512_loadu_pd(&bids[i]);
        best = _mm512_max_pd(best, v);
    }
    double result = _mm512_reduce_max_pd(best);
    for (; i < n; i++) // tail
        result = bids[i] > result ? bids[i] : result;
    return result;
}
```

Gather / Scatter: Price Lookups

- Order books are sparse — the levels you want aren't contiguous in memory
- `_mm512_i32gather_pd` loads 8 non-contiguous doubles from an index vector in one instruction; `_mm512_i32scatter_pd` writes them back the same way

```
#include <immintrin.h>

// gather quotes at 8 arbitrary price-level indices
__m512d gather_levels(const double *book, const int *idx) {
    __m512i vidx = _mm512_loadu_si512((const void*)idx);
    return _mm512_i32gather_pd(vidx, book, sizeof(double));
}
```

Note:

Turns 8 scattered pointer-chasing loads into a single vector instruction
a common pattern in limit-order-book engines.

Why This Matters

- Firms such as Optiver and Jane Street publicly describe tick-to-trade paths measured in tens to low-hundreds of nanoseconds
- SIMD is one of the few “free” levers left at that scale: same algorithm, more data per instruction, no extra branches
- **Typical wins in this style of code (illustrative, workload-dependent)**
 - Branchless AVX-512 masking removes branch mispredictions on the hot path
 - Horizontal reduction turns an $O(n)$ scalar scan into $O(n/8)$ vector ops
 - Gather/scatter collapses sparse random access into one instruction
- None of this replaces algorithmic improvements — it multiplies whatever headroom the algorithm already has

Key Takeaways

- AMX extends the SIMD idea from vectors to whole matrix tiles — one instruction, thousands of MACs
- AVX-512 masks and gather/scatter let you write branchless, cache-friendly code for irregular, latency-critical data like order books
- Same building blocks as the rest of Part 2 — registers, intrinsics, compiler flags — just aimed at matrices and sparse data instead of dense arrays

Resources on AMX & SIMD in Trading

For further information see

- Intel® 64 and IA-32 Architectures Software Developer's Manual — AMX chapter
- Intel Intrinsics Guide:
<https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html>
- “Improving performance with SIMD intrinsics in three use cases” :
<https://stackoverflow.blog/2020/07/08/improving-performance-with-simd-intrinsics-in-three-use-cases/>